EIGER Data Management Standards, Database Access Instructions and Fileserver Documentation

Rev.	Date	Author	Reviewer	Comments
1	July 8, 2021	Simcoe	Eilers	Initial draft: SQL database access
2	July 19, 2021	Simcoe		Added Observation class / S3

1. Purpose

2. Configuring Database Access using AWS Credentials

High-level data products about EIGER fields are stored in a SQL relational database hosed on the Amazon cloud via the Amazon Web Services (AWS) Relational Database Service (RDS). The database is configured in standard Postgresql format, to facilitate communication with a custom python-based API developed by the project team. Raw data products in fits format are also stored on the cloud using Amazon's S3 service. By storing certified project data products on a single, robust cloud server, we eliminate the need to synchronize versions of data products, match reduction runs, and increase visibility and transparency across the geographically distributed team.

Each users' AWS credentials are configured in a way that avoids the need to manage and enter passwords constantly, but it does take a small amount of initial work to set up. We use the Amazon "IAM" credential service, along with Amazon's boto3 python API to build in automatic authentication from python clients. Users install keys on their local host, and then boto3 calls a function that interacts with AWS to generate secure, temporary passwords with very short expiration periods (~10-20 minutes) and automatically uses them to log in without interaction by the user. This section documents steps to set this up on the client side.

2.1. Obtain your credentials

Contact the DB administrator to obtain two keys needed for login: the

"**AWS Access Key ID**" and the

"**AWS Secret Access Key**"

These are NOT the same thing. They should already have been generated when your username/account were created.

Note that the secret key is only made visible one time by AWS when the account is created by the administrator. If you lose your secret key, contact the DB administrator. Your lost key will need to be removed, and a new one can be generated.

2.2. Install Amazon Command Line Interface (CLI) tools

Local copies of the keys must be installed on your machine using Amazon's CLI interface, which you must download and install. Full instructions can be found at this link. Many of our group members will be installing locally on a Mac laptop, and the following commands should download and then install (the ">" is a terminal prompt):

```
> curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
> sudo installer -pkg AWSCLIV2.pkg -target /
```

The second command requires root privileges on your laptop or local machine.

2.3. Configure your local environment

After you have installed the AWS CLI tools (Section 2.2), you need to register the keys obtained from the administrator. From a Terminal prompt, enter:

```
> aws configure
```

You will get several prompts, which should be filled in as follows:

```
AWS Access Key ID [None]: <enter AWS Access Key ID here>
AWS Secret Access Key [None]: <enter AWS Secret Access Key here>
Default region name [None]: us-east-2
Default output format [None]: json
```

The last two fields should be entered as shown when prompted. We may be able to create a European access region if there are connection lags. The default output should be json for compatibility with the eiger python code.

This will create a directory ~/.aws containing some configuration information, and send additional information to AWS to configure the remote end of the connection. You should now be ready to connect to the database.

2.4. Install relevant python libraries

You will need to install the AWS API python library, called boto3:

```
> python -m pip install boto3
```

And also a python library to handle postgresql database connections. We will use the psycopg2 library, which is the most widely-used community standard.

```
> pip install psycopg2
```

Some team members have had more success installing psycopg2 using condo rather than pip, both are supported.

2.5. Test the connection using provided sample script

From a terminal prompt, cd into your [EIGER Git directory]/Database/SQL. Open the file eiger_dbtest.py in an editor, and change the field “USR” to your username for the database. Everything else should remain the same.

Then run:

```
> cd [EIGER GitHub directory]/Database/SQL
> python “eiger_dbtest.py”
```

If the code worked correctly, it should print out a list of the quasars to be targeted with JWST.

2.6. Test credentials with the AWS Fileserver S3

Large fits data files are not stored in the SQL database, as this is more efficient for storing tabulated numerical data than large binary files. It is more convenient to store these on a cloud fileserver, in this case the AWS S3 system. The IAM credentials you set up earlier should also give you access to a special gto1243 file server area on S3.

To test your access, you can run the following:

```
> cd [EIGER GitHub directory]/Database/Fileserver
> python s3_test.py
```

And you should see a printout as follows:

```
[{'Name': 'gto1243', 'CreationDate': datetime.datetime(2021, 6, 2, 2,
50, 31, tzinfo=tzutc())}]
```

This is the GTO1243 file server, named after the EIGER's JWST program handle.

Also, upon success the code will download a FIRE spectrum of ULAS1120 (in fits format) to your working directory from the S3 Fileserver. This file can be deleted after the test is run.

Table 1:	Table 2	Table 3	Table 4	Table 5	Table 6	Table 7	Table 8	Table 9
Galaxies	Quasars	AbsSystems	AbsLines	VoigtComponents	VoigtIons	VoigtTransitions	Observations	Lines
galaxyID	quasarID	absSysID	absLineID	voigtComponentID	voigtIonID	TransitionID	ObsID	lineID
Name	qsoRA	quasarID	absSysID	absSysID	absSysID	lineID	quasarID	Name
QuasarID	qsoDEC	zCentroid	quasarID	zComponent	voigtComponentID	DeltaV_low	Instrument	restWv
RA	emissionRedshift	zCentroidErr	lineID	zComponentErr	ColumnDensity	DeltaV_high	Observatory	oscillatorStrength
DEC	emissionRedshiftErr		observedWave	btherm	ColumnErr		DateObs	gamma
Redshift	absoluteMagnitude		zLine	bthermErr			CalibratedDataFile	ionizationPotential
RedshiftErr	bolometricL		zLineErr	bturb			Filter	
ImpactParameter	nearZoneSize		Wr_sum	bturbErr			Zeropoint	
F606W	EW_Hbeta		Wr_sumerr				Dereddened	
F700W	EWerr_Hbeta		Wr_fit				Reddening	
F814W	FIRE_obsID		Wr_fiterr				Units	
F120W	Xshooter_obsID		N_AODM					
F200W	NIRCAM_obsID		N_AODMerr					
F356W			AODM_flag					
R_eff								
Axis ratio								
PA_Sky								
PA_QSO								
SersicIndex								
M_1400								
StellarMass								
StellarMassError								
SFR								
SFRError								
HaloMass								
HaloMassErr								
EW_5007								
EW_5007err								
EW_6563								
EW_4800								

3. SQL Database Structure

All SQL (Structured Query Language) databases consist of sets of tables containing independent and logically distinct groupings of data, but which relate to each other through cross-referencing. A well-designed set of tables (a.k.a. a "schema") contains no redundant entries, and each row in the table is indexed by a unique integer ID. There are several flavors

of SQL in use, the EIGER database uses the **Postgresql** standard, which has well-supported public libraries in python and other languages.

3.1. EIGER's Tables

EIGER's primary database contains nine tables, whose names and columns are listed in Figure 1, and include:

- **Galaxies**: these are objects that will be detected in the NIRCAM data, and the table contains all measurement data and derived parameters on a per-object basis.
- **Quasars**: A list of the six background quasars, one per field, with relevant physical measurement parameters. This may be fully populated before JWST launches.
- **AbsSystems**: A table of absorption systems, identified only by quasar sightline and redshift
- **AbsLines**: A list of detected individual absorption lines, with quasar, redshift (measured per-line), ion identification, and equivalent width measurements
- **VoigtComponents**: Fit information for absorption systems that are broken into multiple components. Distinct from AbsLines in that the Voigt components are parametric fits. Specified by redshift and Doppler parameter
- **VoigtIons**: Individual transitions associated with a given ion (e.g. CIV). Specified by column density
- **VoigtTransitions**: Individual line IDs, containing information about the identification (e.g. MgII 2796A), as opposed to an Ion which is fitted over all transitions of a multiplet
- **Lines**: Atomic data associated with each line used for analysis, so that the team is using a standardized set of atomic physics
- **Observations**: Descriptions of data obtained for the program, classified by instrument and observatory. Each row contains a link to a reduced data file (and error file, if appropriate) on the S3 Fileserver, or if not, then a placeholder where the reduced file should go.

More tables may be added to the schema as needed, and more columns may easily be added to each table. Nearly all of the tables except for Observations and Galaxies can be completely populated before JWST's launch, with the latter two being tested for population using Mirage simulations pre-launch.

3.2. How tables are cross-referenced

Figure 1 illustrates that each table has in its first column a "primary key" which is a unique integer ID associated with that line in the table. These keys are automatically generated when new rows are added to the table. Relationships between elements of different table are cross-referenced using "external keys," which are extra columns containing primary keys of other tables:

- **Example 1**: A galaxy with galaxyID=100 is located in the field of quasar with quasarID=2 in their respective tables. The table Galaxies therefore has a field called quasarID, so that each row with a galaxy in the field of that quasar has "2" entered in the quasarID field in the Galaxies table.

- **Example 2**: An absorption system is found in the spectrum of a quasar, and you want to access the spectrum for analysis. The AbsSystem table has in each row an entry for a quasarID, and the Observations table also has a quasarID tag. SQL allows you to find all Observations whose quasarID is equal to the quasarID of the AbsSystem, and return links to the reduced data files on the fileserver.

3.3. API for Connecting to the EIGER database

Low-level commands for interacting with the database are contained in an Eiderdb class defined in the file eigerdb.py.

The operational principle centers around opening a connection with a virtual "cursor" into which the user sends ASCII-formatted command or query strings in SQL syntax, and then fetches and returns responses from the DB server.

After importing the eigerdb module, users create a database instance "db" by calling the following method:

```
> db = eigerdb.Eigerdb()
> db.getcursor()
```

At this point the object named db has connected to the cloud and opened a connection to the database.

Any SQL command may then be sent to the database using the query method contained in the database object. Input commands must be strings containing legal SQL syntax.

```
> query_string = 'SELECT name, ra_string, dec_string FROM quasars'
> reply = db.query(query_string)
```

This command should return a list of all the survey quasars by informal name, and their coordinates. The response will be stored in reply as a tuple.

When the connection is no longer needed it should always be shut down using the method

```
> db.close()
```

4. Common SQL Operations / Associated Command Syntax

Only a few very common SQL operations have been captured to date with specific methods. This is I'm part because the eigerdb.query() method captures nearly the full flexibility of the system to operate any command so long as one can generate the right SQL command. There is extensive documentation online about postgresql syntax and users can consult the web for complicated queries.

The most common operations use either:

- [`SELECT/WHERE`] Select entries meeting specified criteria from a table

- [`INSERT INTO`] Add new row entries to an existing table
- [`UPDATE/SET/WHERE`] Modify existing row entries in an existing table
- [`JOIN`] Splice tables together to generate new query responses using relational tags

Two common needs are to list the column names of a given table, and to list the names of all tables in the database. There are special methods to do this in the eigerdb object:

```
> db.tablenames()
> db.columnNames('quasars')
```

The first lists the tables in the database, the second lists all columns in the quasars table.

5. Uploading new Observations and fetching data files from the cloud: the `Observation` class

The code contains an `Observation` class that is stored as a table in the SQL database, but also forms a bridge to the S3 fileserver where the actual data are stored.

To add a new observation to the database, you should create a new `Observation` object, attach relevant metadata describing the instrument and nature of the observations as well as a local name of the data file that will be uploaded to the database.

```
>import Observations
>obs=Observations.Observation(instrument='ACS',observatory='HST',quasa
rID=2,exptime=3600.0,filtername='F775W',datafile='/Users/jwstuser/
data/159_acs_775.fits')
> obs.addToDatabase()
```

```
Connected to the eiger database
Local:  /Users/jwstuser/data/159_acs_775.fits
Remote: HST/ACS/159_acs_775.fits
addToDatabase: All done!
```

This method will upload the local data file to the remote S3 fileserver, and then take all of the metadata you have associated with the observation structure and add it to the SQL database so that other users can search it efficiently.

The remote database has a Filesystem structure that is organized by observatory->instrument. To accommodate database upload of simulated observations, `observatory='Mirage'` is a legal entry but should have an instrument included (e.g. `instrument='NIRCAM'`)

It is extremely useful to have the quasarID attached to every observation, as this tells the database what field a particular observation was targeting.

If you don't remember the quasarIDs, there is a very simple way to query this from the database. We reproduce it here for convenience:

```
In [2]: import eigerdb

In [3]: edb=eigerdb.Eigerdb()

In [4]: edb.getcursor()
Connected to the eiger database

In [5]: edb.query('select id,name from quasars')
Out[5]:
[(1, 'J1030+0524'),
 (2, 'P159-02'),
 (3, 'J1120+0641'),
 (4, 'J0100+2802'),
 (5, 'J1148+5251'),
 (6, 'J0148+0600')]

In [6]: edb.close()
```

So, J1030+0524 has quasarID=1, P159-02 has quasarID=2, etc.